

AER1216: Fundamentals of UAV Assignment #2

Christopher McGale - 1005303841

Feiyang Wu - 1012831205

Clarence Zhang - 1006229591

Question 1: Fixed-Wing UAS Dynamics

(a) State Vector Definition

The state vector $\mathbf{x}$ contains 12 variables that fully describe the aircraft's kinematic and dynamic state:

- **Position** (inertial frame, I):

p_n : North position

p_e : East position

h : Altitude

- **Velocity** (body frame, B):

u : Longitudinal velocity (forward)

v : Lateral velocity (right-wing)

w : Normal velocity (down)

- **Attitude** (Euler angles, from I to B):

ϕ : Roll angle

θ : Pitch angle

ψ : Yaw angle

- **Angular Velocity** (body frame, B):

p : Roll rate

q : Pitch rate

r : Yaw rate

The complete state vector is:

$$\mathbf{x} = [p_n \ p_e \ h \ u \ v \ w \ \phi \ \theta \ \psi \ p \ q \ r]^T \quad (1)$$

(b) Derivatives of Position in Inertial Frame

The derivative of position is velocity, but requires transformation from body frame to inertial frame using the Direction Cosine Matrix (DCM) $\mathbf{R}_I^B$:

$$\dot{\mathbf{p}}^I = \mathbf{R}_I^B \mathbf{v}^B \quad (2)$$

where $\mathbf{R}_B^I = (\mathbf{R}_I^B)^T$ and $\mathbf{v}^B = [u, v, w]^T$.

For a 3-2-1 Euler sequence $(\psi-\theta-\phi)$, the DCM is:

$$\mathbf{R}_I^B = \begin{bmatrix} c_\theta c_\psi & c_\theta s_\psi & -s_\theta \\ s_\phi s_\theta c_\psi - c_\phi s_\psi & s_\phi s_\theta s_\psi + c_\phi c_\psi & s_\phi c_\theta \\ c_\phi s_\theta c_\psi + s_\phi s_\psi & c_\phi s_\theta s_\psi - s_\phi c_\psi & c_\phi c_\theta \end{bmatrix} \quad (3)$$

where $c_x = \cos(x)$, $s_x = \sin(x)$.

In component form:

$$\begin{aligned} \dot{p}_n &= u \cos \theta \cos \psi + v(\sin \phi \sin \theta \cos \psi - \cos \phi \sin \psi) \\ &\quad + w(\cos \phi \sin \theta \cos \psi + \sin \phi \sin \psi) \end{aligned} \quad (4)$$

$$\begin{aligned} \dot{p}_e &= u \cos \theta \sin \psi + v(\sin \phi \sin \theta \sin \psi + \cos \phi \cos \psi) \\ &\quad + w(\cos \phi \sin \theta \sin \psi - \sin \phi \cos \psi) \end{aligned} \quad (5)$$

$$\dot{h} = u \sin \theta - v \sin \phi \cos \theta - w \cos \phi \cos \theta \quad (6)$$

(c) Derivatives of Body Frame Velocity

From Newton's second law in rotating body frame:

$$m\dot{\mathbf{v}}^B + \boldsymbol{\omega}^B \times (m\mathbf{v}^B) = \mathbf{F}^B \quad (7)$$

Rearranging for $\dot{\mathbf{v}}^B$:

$$\dot{\mathbf{v}}^B = \frac{1}{m}\mathbf{F}^B - \boldsymbol{\omega}^B \times \mathbf{v}^B \quad (8)$$

In component form ($\boldsymbol{\omega}^B = [p, q, r]^T$, $\mathbf{v}^B = [u, v, w]^T$):

$$\dot{u} = vr - wq + \frac{1}{m}F_x \quad (9)$$

$$\dot{v} = wp - ur + \frac{1}{m}F_y \quad (10)$$

$$\dot{w} = uq - vp + \frac{1}{m}F_z \quad (11)$$

where $\mathbf{F}^B = [F_x, F_y, F_z]^T$ represents total forces (aerodynamic, gravitational, propulsive) in body frame.

(d) Derivatives of Attitude (Euler Angles)

The relationship between Euler angle rates and body angular rates is:

$$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} 1 & \sin \phi \tan \theta & \cos \phi \tan \theta \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi \sec \theta & \cos \phi \sec \theta \end{bmatrix} \begin{bmatrix} p \\ q \\ r \end{bmatrix} \quad (12)$$

Note the singularity at $\theta = \pm 90^\circ$ (pitch straight up/down), which is a limitation of Euler angles.

(e) Derivatives of Rotation Rate Variables

From Euler's equations of motion for a rigid body:

$$\dot{p} = \frac{1}{J_x} [L + (J_y - J_z)qr] \quad (13)$$

$$\dot{q} = \frac{1}{J_y} [M + (J_z - J_x)pr] \quad (14)$$

$$\dot{r} = \frac{1}{J_z} [N + (J_x - J_y)pq] \quad (15)$$

where $\mathbf{M}^B = [L, M, N]^T$ represents total moments (roll, pitch, yaw) in body frame, and $\mathbf{J} = \text{diag}(J_x, J_y, J_z)$ is the inertia tensor (assumed diagonal for simplicity).

(f) Aerodynamic Forces and Moments

(f.i) Expressions for Forces and Moments

Using non-dimensional aerodynamic coefficients:

Forces:

$$F_{A_x} = \bar{q}S C_X \quad (\text{Axial Force}) \quad (16)$$

$$F_{A_y} = \bar{q}S C_Y \quad (\text{Side Force}) \quad (17)$$

$$F_{A_z} = \bar{q}S C_Z \quad (\text{Normal Force}) \quad (18)$$

Moments:

$$L = \bar{q}Sb C_l \quad (\text{Roll Moment}) \quad (19)$$

$$M = \bar{q}Sc C_m \quad (\text{Pitch Moment}) \quad (20)$$

$$N = \bar{q}Sb C_n \quad (\text{Yaw Moment}) \quad (21)$$

where:

- $\bar{q} = \frac{1}{2}\rho V^2$: Dynamic pressure
- ρ : Air density
- V : Airspeed ($V = \sqrt{u^2 + v^2 + w^2}$)
- S : Wing reference area
- b : Wingspan
- c : Mean aerodynamic chord

(f.ii) Methods for Modelling Aerodynamic Coefficients

1. Look-up Tables from Wind Tunnel Data or CFD:

- **Description:** Coefficients stored as functions of key variables ($\alpha, \beta, p, q, r, \delta_e, \delta_a, \delta_r, M$)
- **Applications:** High-fidelity simulation, flight testing, validation for nonlinear aircraft in wide flight envelopes
- **Implementation:** Real-time interpolation within pre-computed tables during simulation

2. Linearized (Stability Derivative) Models:

- **Description:** Linear expansions around trim conditions:

$$C_m \approx C_{m_0} + C_{m_\alpha} \alpha + C_{m_q} \frac{qc}{2V} + C_{m_{\delta_e}} \delta_e \quad (22)$$

- **Applications:** Control system design, stability analysis, basic simulation for small perturbations
- **Implementation:** Direct computation using linear algebraic equations in simulation

3. Component Build-Up (Semi-Empirical) Methods:

- **Description:** Software tools (DATCOM, AVL, ASWING) estimate coefficients from geometric properties
- **Applications:** Preliminary design phase for conventional UAS
- **Implementation:** Offline coefficient generation followed by table lookup in simulation

Question 2a: Fixed-Wing PID Design

For all our PID controllers we decided to use an architecture known as 'derivative on measurement'. This allows us to avoid 'derivative kick' when a command changes abruptly (e.g. increasing altitude to from 0-100m). Note that because we use DoM, we multiply the measurement value by -1 to reduce control action rather than amplify it. This is widely used in industry to allow for a smooth response to step commands, which is likely how we will design the mission state machine given the final project requirements.

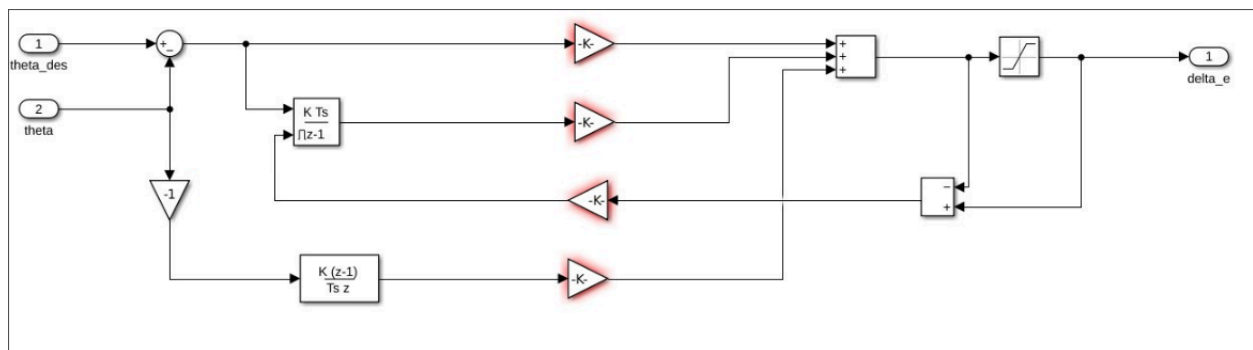
For all inner loop controllers we set a sample time of 0.01s allowing for faster correction of high-frequency disturbances and sufficient control bandwidth to stabilize the aircraft dynamics before the outer loop updates.

We set a sample time of 0.1s for the outer loop because the inner controllers require multiple cycles to settle before new reference commands are issued, preventing control interaction and maintaining stability margins between the inner and outer control layers.

Because of these sample times (which are more realistic to how a physical controller would operate) we use discrete integrals and derivatives throughout to avoid any approximation errors.

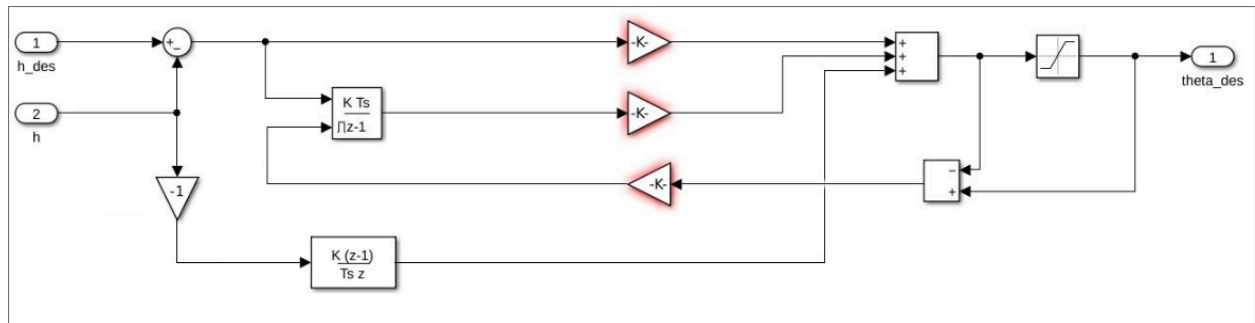
Anti-windup protection is implemented by comparing the saturated and unsaturated controller outputs. If saturation occurs (difference $\neq 0$), the integrator is externally reset to prevent wind-up.

i) Pitch Attitude Controller



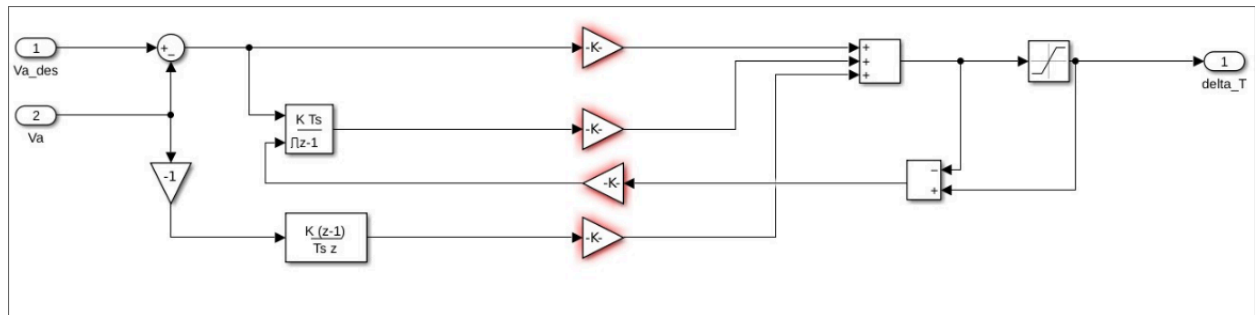
The pitch attitude controller is a PID that commands elevator deflection (δ_e) to regulate pitch angle (θ). It receives desired pitch from either the altitude-hold or steady-climb outer loop and uses derivative on measurement to smooth transient response without kick.

ii) Altitude Hold Controller



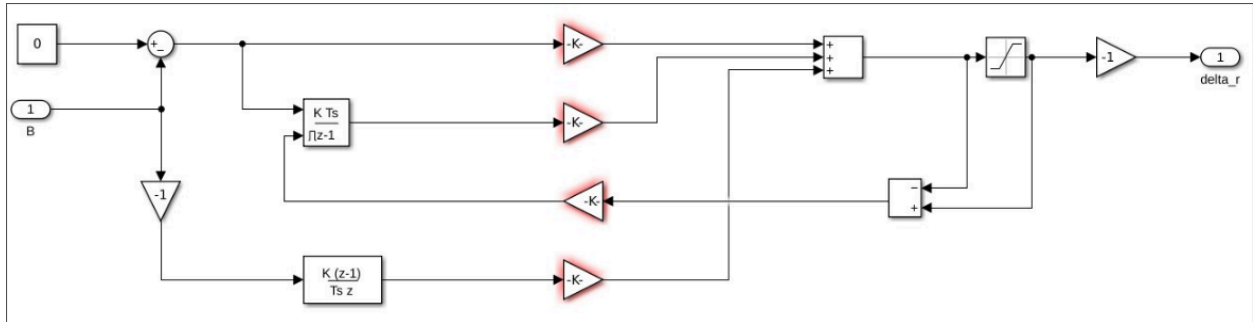
The altitude-hold controller forms the outer loop for pitch control, commanding desired pitch angle (θ_{des}) based on altitude error. It operates as a PI controller (no derivative needed at the outer loop level) and saturates pitch commands to $\pm 20^\circ$ to respect aircraft limits and maintain control authority for the pitch controller.

iii) Airspeed Controller



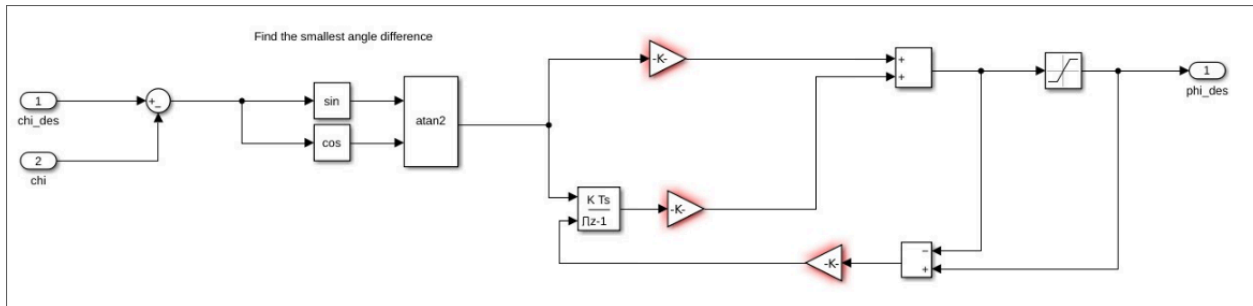
The airspeed controller regulates velocity (V_a) by commanding throttle (δ_T) in the range 0–100%. It operates as a standard PID with derivative on measurement and prevents integrator wind-up by saturating at the aircraft's throttle limits. The airspeed controller receives either a direct airspeed command from the mission state machine (cruise mode) or an airspeed setpoint computed by the steady-climb controller (climb/descent mode). This is toggled via a switch block.

iv) Zero-Sideslip Angle Controller



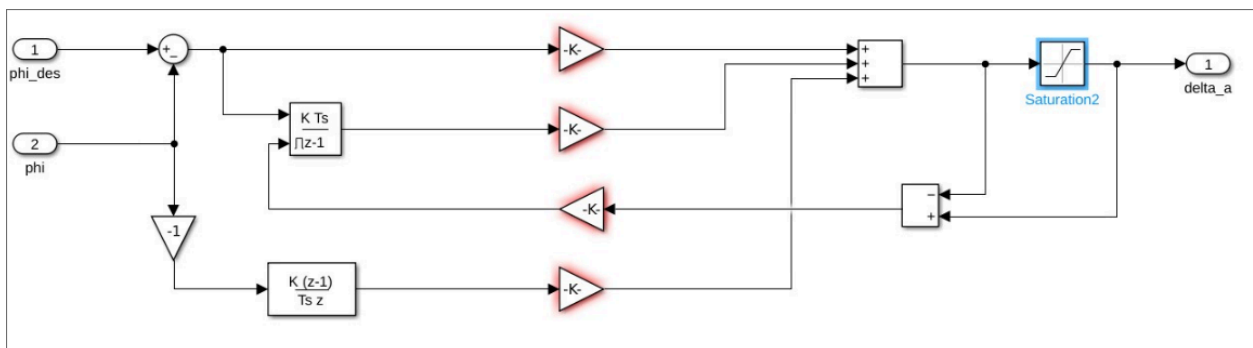
The sideslip controller maintains coordinated flight by commanding rudder deflection (δ_r) to regulate sideslip angle (β) to zero. Unlike pitch and altitude, this controller has no outer loop since the desired sideslip is always zero. It operates as a PI controller and prevents integrator saturation when the rudder hits its $\pm 20^\circ$ limits.

v) Heading Angle Controller



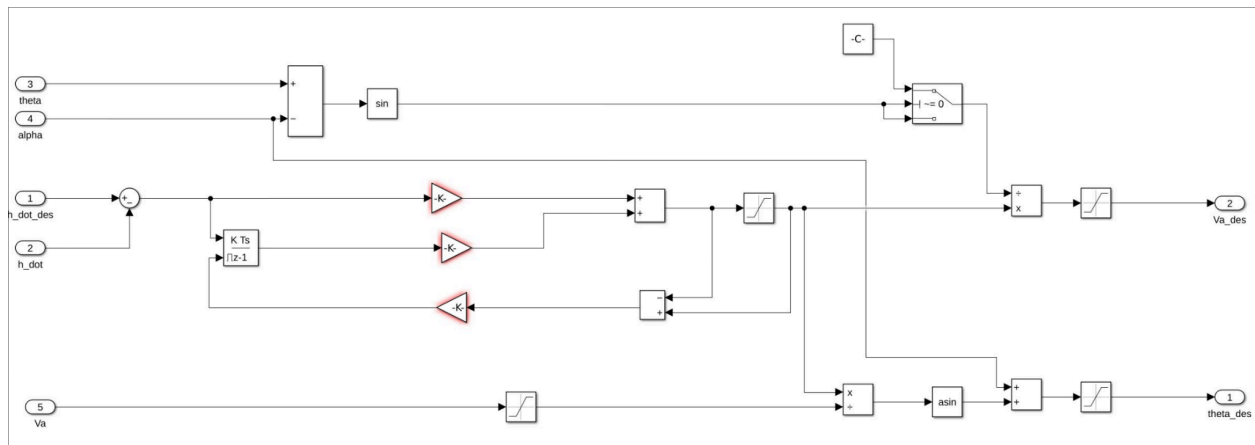
The heading controller forms the outer loop for roll control, commanding desired roll angle (ϕ_{des}) based on heading error (χ). It operates as a PI controller with heading wrap-around logic using atan2 to ensure the aircraft turns via the shortest path across the $360^\circ/0^\circ$ boundary.

Roll Attitude Controller



The roll attitude controller is a PID that commands aileron deflection (δ_a) to regulate roll angle (ϕ). It receives the desired roll from the heading controller and uses derivative on measurement for smooth, stable response.

vi) Steady Climb Controller



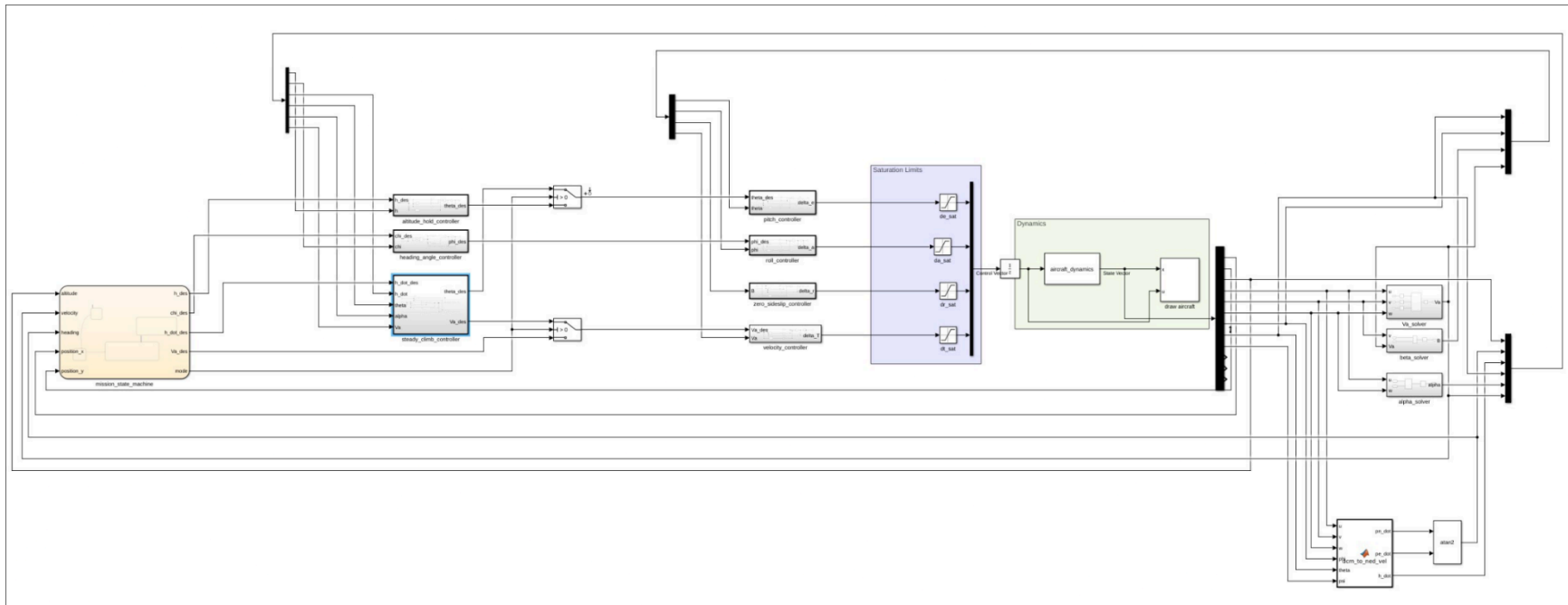
The steady-climb controller regulates vertical climb rate ($\dot{h}$) by simultaneously commanding both pitch angle and airspeed setpoints to their respective inner loop controllers. It operates as a PI controller and uses the kinematic relationship $\dot{h} = V_a \sin(\gamma)$ to convert climb rate commands into coordinated pitch and airspeed setpoints.

To avoid a division by zero errors we saturate V_a between $e-5$ and the max airspeed. Because the controller should work for climb and descent, γ could be positive or negative. We never plan to have a negative V_a so this should suffice. To adjust for this instead of a saturation, we used a switch block set to use a constant of $e-5$ if γ was ever 0 so that the controller would not crash from the error.

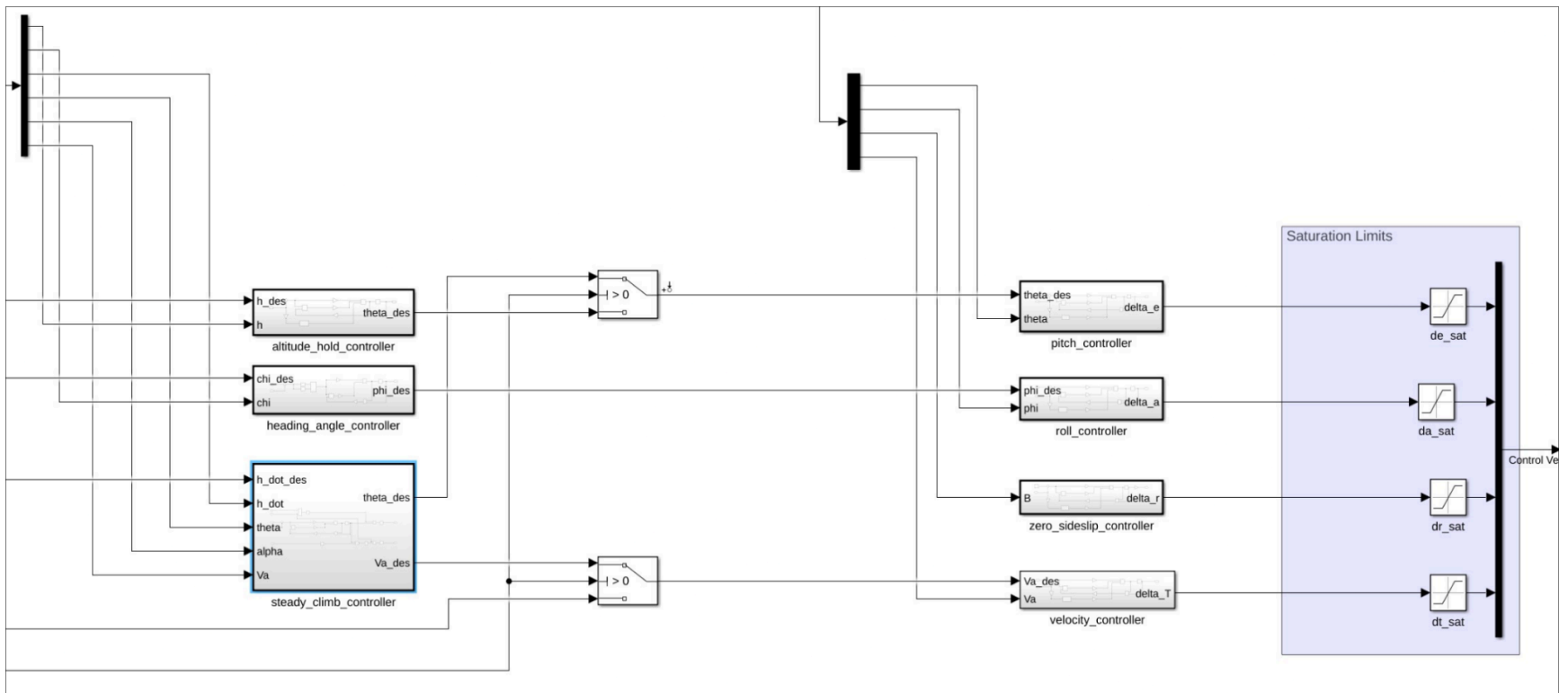
Both pitch and airspeed outputs are saturated appropriately: airspeed to aircraft speed limits and pitch to $\pm 20^\circ$ to maintain control authority.

The steady-climb controller coordinates pitch and airspeed setpoints, feeding desired values to the pitch and airspeed controllers respectively. When steady-climb mode is active, this controller overrides the direct mission state machine airspeed command via the switch block described in (iii).

vii) Full Control Block



This is our working copy of the fixed wing controller so far. It includes all 7 of the PID controllers described in this assignment as well as a state machine for the mission plan and implemented dynamics blocks which convert our state variables to those needed by the 7 controllers. The state machine provides desired values to our outer loop of controllers (altitude hold, heading angle, steady climb) and a mode setting for altitude hold vs climb which controls the switch outputs between the inner and outer loops. Mode switching between altitude-hold and steady-climb is implemented via a Simulink switch block that toggles between outer-loop setpoint commands. The outer loops send their results as the desired input to the inner loop controllers, which command the thrust and control surfaces directly.



A closer view of the controllers so that you can see the connections.

Q2b) PID Tuning Strategy

To provide an initial measurement we can use the guides below or we can use an algorithmic solver like Simulinks Control System Designer or other built in matlab functions. These are quite good for Single Input Single Output models (SISO) but can struggle heavily with Multi Input Multi Output models like we described above.

Start by tuning the gains sequentially in order: $P \rightarrow D \rightarrow I$. This approach minimizes parameter interactions and allows systematic refinement based on observable behavior.

Proportional Gain (K_p)

The proportional term is the foundation and should be tuned first. It scales the control response proportionally to the error magnitude and direction.

Initial Estimation: A reasonable starting point is $K_p \approx 1 / (\text{system gain})$.

Tuning: Set K_i and K_d to zero. Apply a step change to the setpoint and gradually increase K_p until the system responds near the target at reasonable speed without excess oscillation. Complete when reaching approximately 90-95% of the setpoint.

Derivative Gain (K_d)

The derivative term provides damping by opposing the rate of change of error. This reduces overshoot and improves settling time.

Initial Estimation: Start with K_d approximately one-tenth of the system's time constant τ (the time to reach ~63% of final value after a step input).

Tuning: Keep K_p fixed and increase K_d gradually from a small initial value. With each increase, apply a step change and observe the overshoot. Tuning is complete when the system approaches the setpoint with minimal overshoot and settles smoothly. Minor adjustments to K_p may be needed if the overall response speed has changed.

Integral Gain (K_i)

The integral term accumulates error over time and eliminates steady-state offset. P control alone cannot eliminate steady-state error because it requires a permanent small error to produce a constant output; integral action solves this by accumulating the effect of even tiny persistent errors.

Initial Estimation: A conservative starting point is $K_i \approx K_p / (2 \times \tau)$.

Tuning: Keep Kp and Kd fixed. Start with a small Ki value and apply a setpoint change or disturbance. After settling, check if the system stabilizes exactly at the setpoint or slightly below. Fine tuning requires analysis of if the system is over vs undershooting and correcting.

Final Refinement

After tuning all three gains, make minor adjustments to P and D based on overall system performance. Increases to Ki sometimes require small reductions in Kp or Kd to maintain stability.

Question 2c: Specific Tuning

Loop Hierarchy

Our control system has inner loops (velocity, pitch, sideslip, roll) that receive setpoint commands from outer loops (heading, altitude, steady climb). **Tuning order:** Velocity → Pitch → Sideslip → Roll → Steady-Climb → Altitude Hold → Heading Angle. This makes the most sense to us because it lets us naturally test the phases of our mission plan in order (climb to height, hold altitude, coordinated turn), while keeping an inner → outer loop pattern which allows for decreased coupling

Inner Loops

For each inner loop, use Control System Designer to estimate baseline gains from step response analysis, then validate in full simulation. For each controller we'll ensure the following criteria before moving on.

- Steady-state error < 1%
- Overshoot ≤ 10% and settling time within 2× target
- Damped response (≤ 2–3 oscillations before settling)

Roll-Pitch Coupling

If roll response becomes sluggish or oscillates when pitch control is simultaneously active, increase roll damping to decouple the interaction.

Pitch-Velocity Coupling (Steady-Climb)

The steady-climb controller couples pitch and velocity through $\dot{h} = V_a \sin(\gamma)$. To mitigate:

1. Tune pitch and velocity independently first
2. Activate steady-climb only after both are stable
3. Start steady-climb gains conservatively

4. Monitor pitch and velocity errors simultaneously during testing; if coupling appears, reduce steady-climb gains and increase pitch damping
5. Verify integrator resets when pitch or throttle saturate

Sideslip-Heading Coordination

During turns, the zero-sideslip controller (rudder) must coordinate with the heading controller (roll). For the case of the final project we can accept small transient sideslip ($\pm 2\text{--}3^\circ$) during coordinated turns. Verify sideslip returns to $< 1^\circ$ within 2 seconds post-turn and that heading control remains effective. If sideslip exceeds $\pm 3^\circ$ or doesn't settle, increase roll damping.

Outer Loops

Once the inner loops (velocity, pitch, roll, sideslip) are stable, activate the outer loops in order: steady-climb $\rightarrow$ altitude-hold $\rightarrow$ heading angle.

Steady-Climb: Coordinates velocity and pitch setpoints. Tune as PI. Start conservatively and increase gains until climb rate error settles within target. Monitor pitch and velocity errors simultaneously. If coupling appears, reduce steady-climb gains and increase pitch damping.

Altitude-Hold: Commands pitch setpoint to the pitch controller. Tune as PI (no derivative). Start conservatively and increase gains until altitude error settles within target. Verify that pitch commands stay within $\pm 20^\circ$ saturation.

Heading Angle: Commands desired roll angle to the roll controller (inner loop). Tune as PI with heading wrap-around logic (atan2) to ensure shortest-path turns. Verify coordinated turns by monitoring both heading and roll simultaneously. If sideslip grows during turns, increase roll damping (K_d) in the inner loop.

Mode Transitions: When switching between altitude-hold and steady-climb (both commanding pitch), pre-load the new controller's integrator with a proportionally scaled value from the previous integrator to ensure smooth transitions. Monitor pitch and throttle; transients should be $< 2^\circ$ and $< 5\%$ respectively, settling within 2 seconds.

Performance Metrics

Good control performance is defined by:

1. **Path Tracking:** 3D Euclidean distance error (RMS and peak) between intended and actual flight paths
2. **Individual Loop Performance:**
 - Velocity: Settle $< 5\text{s}$, overshoot $< 10\%$, error $< 1\text{ m/s}$
 - Pitch: Settle $< 3\text{s}$, overshoot $< 10^\circ$, error $< 1^\circ$
 - Roll: Settle $< 2\text{s}$, overshoot $< 5^\circ$, error $< 1^\circ$

- Heading: Settle < 5s, overshoot < 10°, error < 1°
- Altitude: Settle < 5s, overshoot < 2%, error < 1 m
- 3. **Altitude-Hold Constraints:** Angle of attack < 10°, elevator deflection < 15°, throttle < 30%
- 4. Angle of Attack Protection: Post-flight validation—compute $\alpha = \theta - \arctan(\dot{h}/V_a)$. If α exceeds 10°, reduce pitch saturation limit to $\pm 15^\circ$ or increase airspeed setpoint.

Robustness Validation

Test the tuned controller against:

- **Wind gusts** (3 m/s perpendicular to flight path): Aircraft deviation < 10 m horizontal, < 5 m vertical, settling within 5s
- **Sensor noise** (± 0.5 m/s airspeed, ± 0.5 m altitude, $\pm 0.5^\circ$ pitch): Control outputs smooth without > 10 Hz chatter
- **Parameter uncertainty** ($\pm 15\%$ mass, $\pm 10\%$ drag coefficient, $\pm 20\%$ pitch inertia): Climb rate error < 1 m/s, throttle adjustment < 15%, settle time increase < 30%
- **Actuator delays** (75 ms): Aircraft settles within 5s with < 15% overshoot, no oscillatory divergence

If any test fails, reduce K_p/K_d in the affected loop and re-test.

Iteration and Documentation

Maintain a tuning log recording: iteration number, gains modified, test scenario, metrics (settling time, overshoot, error), pass/fail status, and next steps. This enables backtracking and identifies gain sensitivity patterns.

Symbol conventions

Frames and axes (2D planar model):

Inertial frame: we use the y – z plane. $+y$: horizontal to the right; $+z$: vertical upward; gravity acts along $-z$.

Body frame: the *total thrust* acts along the body's $+z_b$ axis; pitch angle ϕ rotates the body about the out-of-plane axis (nose-up $\phi > 0$).

Inputs and geometric parameters:

Individual propeller thrusts (inputs): $T_1(t), T_2(t)$ with $T_1, T_2 \geq 0$.

Total thrust and pitch moment:

$$u_T \triangleq T_1 + T_2, \quad u_M \triangleq l(T_2 - T_1),$$

where l is the arm length from the vehicle's CoM to each propeller center.

Aerodynamic drag and wind:

A constant wind blows from left to right with speed v_∞ . The *drag magnitude* is given by

$$F_d = \frac{1}{2} C_d \rho A (v_{\text{rel}})^2 = k (v_{\text{rel}})^2, \quad k = \frac{1}{2} C_d \rho A,$$

where v_{rel} is air velocity relative to the object. The assignment specifies no drag in the z direction and no rotational drag.

Q3(a) Derivation of the nonlinear first-order model

Problem restatement

The problem considers 2D motion in the x – z plane with position $(x(t), z(t))$ and pitch angle $\theta(t)$, where the inputs are individual nonnegative thrusts $T_1(t), T_2(t)$. A constant wind blows from left to right with speed v_∞ and produces aerodynamic drag, with $k = \frac{1}{2} C_d \rho A$ defined as the drag coefficient. Drag in the z direction and rotational drag are neglected, and the task is to take the thrusts as inputs, $[x(t), z(t)]$ as outputs, and derive a nonlinear first-order system.

Forces and moments in the x – z plane

Thrust decomposition: the total thrust $u_T = T_1 + T_2$ points along $+z_b$; in the inertial frame it contributes horizontal component $-u_T \sin \theta$ (to the left for $\theta > 0$) and vertical component $+u_T \cos \theta$. Hence (without wind) the translational equations are

$$m\ddot{x} = -u_T \sin \theta, \quad m\ddot{z} = -mg + u_T \cos \theta.$$

Pitch dynamics: about the CoM with inertia J , the net pitch moment is $u_M = l(T_2 - T_1)$, giving

$$J\ddot{\theta} = u_M = l(T_2 - T_1).$$

Sign convention: positive $T_2 - T_1$ produces positive $\ddot{\theta}$.

Wind drag in x-axis: magnitude $k(v_{\text{rel}})^2$ with v_{rel} the air velocity relative to the vehicle. With wind from left to right, define $v_{\text{rel}} = v_{\infty} - \dot{x}$. To encode direction (taking positive force as pushing the vehicle to the right), use the signed form

$$F_{d,x} = k(v_{\infty} - \dot{x})|v_{\infty} - \dot{x}|,$$

which is positive (pushes right) when $v_{\infty} > \dot{x}$ and negative when $\dot{x} > v_{\infty}$.

Second-order equations of motion in assignment symbols

Combining the above,

$$\begin{cases} m\ddot{x} = -(T_1 + T_2)\sin\theta + k(v_{\infty} - \dot{x})|v_{\infty} - \dot{x}|, \\ m\ddot{z} = -mg + (T_1 + T_2)\cos\theta, \\ J\ddot{\theta} = l(T_2 - T_1). \end{cases}$$

The first two lines are Newton's equations in the inertial frame with thrust decomposition and the assignment's drag; the third line is the pitch (Euler) equation.

Nonlinear first-order state-space

We use thrusts as inputs and $[x, z]$ as outputs. Define:

State vector (6-state planar model):

$$x(t) = [x \quad z \quad \theta \quad \dot{x} \quad \dot{z} \quad \dot{\theta}]^T.$$

Input vector:

$$u(t) = \begin{bmatrix} T_1(t) \\ T_2(t) \end{bmatrix}.$$

Output vector:

$$y(t) = \begin{bmatrix} x(t) \\ z(t) \end{bmatrix}.$$

Let $v_{\text{rel}} = v_{\infty} - \dot{x}$. Then the *first-order model* is

$$\begin{aligned}
\dot{x}_1 &= \dot{x}, \\
\dot{x}_2 &= \dot{z}, \\
\dot{x}_3 &= \dot{\theta}, \\
\dot{x}_4 &= \frac{-(T_1 + T_2)\sin\theta + kv_{\text{rel}}|v_{\text{rel}}|}{m}, \\
\dot{x}_5 &= \frac{-mg + (T_1 + T_2)\cos\theta}{m}, \\
\dot{x}_6 &= \frac{l(T_2 - T_1)}{J}, \\
y &= \begin{bmatrix} x \\ z \end{bmatrix}.
\end{aligned}$$

This is the requested *nonlinear first-order system* $\dot{x} = f(x, u), y = g(x, u)$ with thrust inputs and (x, z) outputs.

Q3(b) Equilibrium points

Problem restatement

The problem defines stationary explicitly as $\dot{x}(t) = 0$. That is, all state derivatives are zero at equilibrium. Find all equilibrium points (x_0, u_0, y_0) where the quadrotor is stationary.

Dynamics used

The 2-D planar quadrotor model (world x - z plane, pitch θ):

$$\begin{aligned}
m\ddot{x} &= -U_T\sin\theta + F_{d,x}, & F_{d,x} &= k(v_\infty - \dot{x})|v_\infty - \dot{x}|, \\
m\ddot{z} &= -mg + U_T\cos\theta, \\
J\ddot{\theta} &= l(T_2 - T_1).
\end{aligned}$$

Stationary equilibrium conditions

Let the equilibrium state and inputs be

$$\mathbf{x}_o = [x_0, z_0, \theta_0, 0, 0, 0]^\top, \quad \mathbf{u}_o = [T_{1o}, T_{2o}]^\top,$$

where x_0, z_0 are constant (arbitrary) positions and $\dot{x} = \dot{z} = \dot{\theta} = 0$. Substituting $\dot{x} = 0$, $\ddot{x} = \ddot{z} = \ddot{\theta} = 0$ into the equations above gives three *algebraic balance equations*:

1. Horizontal force balance

$$0 = -U_{T0}\sin\theta_0 + kv_\infty^2 \Rightarrow U_{T0}\sin\theta_0 = kv_\infty^2.$$

2. Vertical force balance

$$0 = -mg + U_{T0}\cos\theta_0 \Rightarrow U_{T0}\cos\theta_0 = mg.$$

3. Pitch moment balance

$$0 = l(T_{2o} - T_{1o}) \Rightarrow T_{2o} = T_{1o}.$$

Solve for the equilibrium orientation and thrusts

From horizontal and vertical force balance:

$$\tan\theta_0 = \frac{kv_\infty^2}{mg} \Rightarrow \theta_0 = \arctan\left(\frac{kv_\infty^2}{mg}\right),$$

and

$$U_{T0} = \sqrt{(mg)^2 + (kv_\infty^2)^2} \Rightarrow U_{T0} = \sqrt{(mg)^2 + (kv_\infty^2)^2}.$$

From pitch moment balance and $U_{T0} = T_{1o} + T_{2o}$ with $T_{1o}, T_{2o} \geq 0$, symmetry gives:

$$T_{1o} = T_{2o} = \frac{U_{T0}}{2}.$$

Equilibrium state, input, and output

State (any fixed position, zero velocities/rates):

$$\mathbf{x}_o = [x_0, z_0, \theta_0, 0, 0, 0]^\top, \quad x_0, z_0 \in \mathbb{R} \text{ arbitrary}.$$

Input (symmetric thrusts):

$$\mathbf{u}_o = [T_{1o}, T_{2o}]^\top = \left[\frac{U_{T0}}{2}, \frac{U_{T0}}{2} \right]^\top.$$

Output:

$$\mathbf{y}_o = [x_0, z_0]^\top.$$

Result

$$\begin{aligned} \text{Equilibrium (stationary): } \mathbf{x}_o &= [x_0, z_0, \theta_0, 0, 0, 0]^\top, \quad \mathbf{u}_o = \left[\frac{U_{T0}}{2}, \frac{U_{T0}}{2} \right]^\top, \quad \mathbf{y}_o = [x_0, z_0]^\top, \\ \theta_0 &= \arctan\left(\frac{kv_\infty^2}{mg}\right), \quad U_{T0} = \sqrt{(mg)^2 + (kv_\infty^2)^2}, \quad T_{1o} = T_{2o} = \frac{U_{T0}}{2}. \end{aligned}$$

Equilibrium points existence discussion

In terms of orientation and thrust magnitudes, the solution is *unique* for given (m, g, k, v_∞) : there is one θ_0 and one U_{T0} that satisfy the two force balances, and symmetry forces $T_{1o} = T_{2o}$.

However, because the problem does not constrain the absolute position, any constant pair (x_0, z_0) is allowed. Therefore there are infinitely many equilibrium points overall (a continuum), parameterized by (x_0, z_0) .

Q3(c) Linearization of the x -channel with input $u(t) = \theta(t)$ about the equilibrium point

Problem restatement

Now treat *pitch angle as the new input* while the total thrust is auto-trimmed:

An attitude controller is designed such that $\theta(t)$ can be chosen as our input, also $(T_1(t) + T_2(t)) = \frac{mg}{\cos\theta(t)}$. Linearize the x -dynamics about the equilibrium found in (b) with input $u(t) = \theta(t)$ and output $x(t)$. Express in $\delta\dot{x} = A\delta x + b\delta u$, $\delta y = c^\top \delta x$ form. Also note that state, input, and output have changed compared to (a)(b).

We will therefore linearize only the x -channel (SISO: input θ , output x)

Nonlinear x -dynamics under the given inner-loop constraint

From part (a), the x -equation (with drag only along x) is

$$m\ddot{x} = -(T_1 + T_2)\sin\theta + F_{d,x}, \quad F_{d,x} = k(v_\infty - \dot{x})|v_\infty - \dot{x}|.$$

Insert the constraint $(T_1 + T_2) = \frac{mg}{\cos\theta}$ (vertical channel auto-compensates gravity):

$$\ddot{x} = -g\tan\theta + \frac{k}{m}(v_\infty - \dot{x})|v_\infty - \dot{x}|.$$

Linearization point (from b)

For the stationary equilibrium: $\dot{x}_0 = 0$ and $\ddot{x}_0 = 0$.

$$0 = -g\tan\theta_0 + \frac{k}{m}v_\infty^2 \Rightarrow \tan\theta_0 = \frac{kv_\infty^2}{mg}.$$

Choose state, input, output and compute the Jacobians

Use the minimal x -channel state:

$$\delta x = \begin{bmatrix} \delta x_1 \\ \delta x_2 \end{bmatrix} = \begin{bmatrix} x - x_0 \\ \dot{x} - \dot{x}_0 \end{bmatrix} = \begin{bmatrix} x - x_0 \\ \dot{x} \end{bmatrix}, \quad \delta u = \delta\theta = \theta - \theta_0, \quad \delta y = \delta x_1.$$

Let

$$f(\dot{x}, \theta) = -g\tan\theta + \frac{k}{m}(v_\infty - \dot{x})|v_\infty - \dot{x}|.$$

Then $\delta \dot{x}_1 = \delta x_2$, and

$$\delta \dot{x}_2 = \underbrace{\frac{\partial f}{\partial \dot{x}} \Big|_0}_{\tilde{a}_{21}} \delta x_2 + \underbrace{\frac{\partial f}{\partial \theta} \Big|_0}_{\tilde{b}_2} \delta \theta.$$

Partial derivatives at the equilibrium:

$$\frac{\partial f}{\partial \theta} = -g \sec^2 \theta \Rightarrow \frac{\partial f}{\partial \theta} \Big|_0 = -g \sec^2 \theta_0$$

Let $r = v_\infty - \dot{x}$, $s(r) = r|r|$. Then

$$\frac{\partial}{\partial \dot{x}} \left[\frac{k}{m} s(r) \right] = \frac{k}{m} s'(r) \cdot (-1).$$

At the chosen equilibrium, $r_0 = v_\infty > 0 \Rightarrow s(r) = r^2$, $s'(r) = 2r$. Hence

$$\frac{\partial f}{\partial \dot{x}} \Big|_0 = -\frac{2kv_\infty}{m}.$$

Final linear model (state-space)

$$\boxed{\begin{aligned} \delta \dot{x} &= \underbrace{\begin{bmatrix} 0 & 1 \\ 0 & -\frac{2kv_\infty}{m} \end{bmatrix}}_{\tilde{A}} \delta x + \underbrace{\begin{bmatrix} 0 \\ -g \sec^2 \theta_0 \end{bmatrix}}_{\tilde{b}} \delta u, \\ \delta y &= \underbrace{\begin{bmatrix} 1 \\ 0 \end{bmatrix}^\top}_{\tilde{c}^\top} \delta x, \quad \text{with } \tan \theta_0 = \frac{kv_\infty^2}{mg}. \end{aligned}}$$

Q4(d) PID design (plus an LQI alternative)

Model (from Q3(c) linearization)

Around the stationary equilibrium of Q3(b), with input $u(t) = \theta(t)$ and output $y(t) = x(t)$, the x -channel linearized model is

$$\delta \dot{x} = \underbrace{\begin{bmatrix} 0 & 1 \\ 0 & -\frac{2kv_\infty}{m} \end{bmatrix}}_{\tilde{A}} \delta x + \underbrace{\begin{bmatrix} 0 \\ -g \sec^2 \theta_0 \end{bmatrix}}_{\tilde{B}} \delta \theta, \quad \delta y = \underbrace{[1 \quad 0]}_{\tilde{C}} \delta x,$$

with

$$\tan \theta_0 = \frac{kv_\infty^2}{mg}, \quad d := \frac{2kv_\infty}{m}, \quad a := g \sec^2 \theta_0.$$

Equivalently: $\delta\ddot{x} + d\delta\dot{x} = -a\delta\theta$.

A reference $x_{\text{des}}(t)$ will be tracked. Define the errors

$$e := x_{\text{des}} - x, \quad \dot{e} = \dot{x}_{\text{des}} - \dot{x}, \quad \ddot{e} = \ddot{x}_{\text{des}} - \ddot{x}.$$

Transform function

Perform Laplace transforms on both sides of $\delta\ddot{x} + d\delta\dot{x} = -a\delta\theta$:

$$s^2X(s) + dsX(s) = -a\Theta(s)$$

Hence

$$G(s) = \frac{X(s)}{\Theta(s)} = \frac{-a}{s(s+d)}$$

Plug in a and d into $G(s)$:

$$G(s) = \frac{X(s)}{\Theta(s)} = \frac{-(g + \frac{k^2 v_{\infty}^4}{m^2 g})}{s(s + \frac{2kv_{\infty}}{m}s)}$$

PID controller design

Design objective (second-order target)

Choose a damping ratio $\zeta \in [0.7, 1]$ and a natural frequency $\omega_n > 0$ to meet rise, settling and overshoot goals. The target error dynamics are

$$\ddot{e} + 2\zeta\omega_n\dot{e} + \omega_n^2 e = 0.$$

Computed-torque (feedback linearizing) PID mapping to θ

Use the plant model to cancel the linear drag and impose the desired error dynamics.

Start from $\delta\ddot{x} + d\delta\dot{x} = -a\delta\theta$.

Command

$$\boxed{\delta\theta = \frac{-d\delta\dot{x} + v}{a}} \Rightarrow \delta\ddot{x} = -v.$$

To make $\ddot{e} + 2\zeta\omega_n\dot{e} + \omega_n^2 e + K_i \int e dt = 0$, set

$$\boxed{v = -\ddot{x}_{\text{des}} - 2\zeta\omega_n\dot{e} - \omega_n^2 e - K_i \int e dt}.$$

Putting it together, the *PID law in terms of θ* is

$$\delta\theta = \frac{-d\delta\dot{x} - \ddot{x}_{\text{des}} - 2\zeta\omega_n\dot{e} - \omega_n^2 e - K_i \int e dt}{a}$$

$$\delta\theta = \underbrace{-\frac{\omega_n^2}{a} e}_{\text{P}} - \underbrace{\frac{2\zeta\omega_n}{a} \dot{e}}_{\text{D}} - \underbrace{\frac{K_i}{a} \int e dt}_{\text{I}} - \underbrace{\frac{d}{a} \delta\dot{x}}_{\text{Damping Compensation}} - \underbrace{\frac{\ddot{x}_{\text{des}}}{a}}_{\text{Feedforward}}$$

and the *absolute command* is $\theta_{\text{ref}} = \theta_0 + \delta\theta$.

How to pick the gains

Choose the value of $\zeta \in [0.7, 1]$. Then pick ω_n from the desired settling time T_s with $T_s \approx 4/(\zeta\omega_n)$. Start with $K_i = 0$ (PD only). Tune ω_n (and ζ if needed) to meet rise/overshoot. Then open K_i gradually to remove any steady-state error. A common rule of thumb is to place the integral pole around 3 ~ 5 times lower than the closed-loop bandwidth, e.g. $K_i \approx \alpha\omega_n^2$ with $\alpha \in [0.2, 0.4]$.

Implementation details that must be

Anti-windup: limit θ_{ref} and use back-calculation or conditional integration on the $\int e$ path.

Derivative filtering: if you realize $\dot{e}$ numerically, filter it (first-order filter cutoff 3 ~ 10 $\times$ the closed-loop bandwidth).

Loop separation: keep the inner attitude/vertical loops $\geq 3 - 5 \times$ faster than this outer x loop, consistent with the assumption used in Q3(c).

Alternative technique — LQI (LQR with integral)

LQI definition

LQI is LQR + an integral of the tracking error.

We augment one state $z(t) = \int (r - y)dt$. The regulator then minimizes a quadratic cost over the augmented state $\xi := \begin{bmatrix} \delta x \\ z \end{bmatrix}$ to obtain a state-feedback law

$$u(t) = -K_x \delta x(t) - K_i z(t), \quad \text{with } \dot{z}(t) = r(t) - y(t),$$

Where $r(t)$ is the reference value which is the target of output, $y(t)$ is the output value which is the the current measured position deviation.

Augmented plant for LQI

Define the integral state

$$z = \int (r - y)dt = \int (r - C\delta x)dt, \quad \dot{z} = r - C\delta x.$$

Form the augmented system

$$\underbrace{\begin{bmatrix} \delta \dot{x} \\ \dot{z} \end{bmatrix}}_{\check{\xi}} = \underbrace{\begin{bmatrix} A & 0 \\ -C & 0 \end{bmatrix}}_{\check{A}_a} \underbrace{\begin{bmatrix} \delta x \\ z \end{bmatrix}}_{\check{\xi}} + \underbrace{\begin{bmatrix} B \\ 0 \end{bmatrix}}_{\check{B}_a} u + \underbrace{\begin{bmatrix} 0 \\ 1 \end{bmatrix}}_{\check{E}_a} r.$$

LQI optimization and control law

Choose quadratic weights $Q_a \geq 0$ and $R > 0$, and solve the continuous algebraic Riccati equation

$$A_a^T P + P A_a - P B_a R^{-1} B_a^T P + Q_a = 0.$$

Set

$$K_a = R^{-1} B_a^T P = [K_x \quad K_i].$$

Then the LQI law is

$$u = -K_x \delta x - K_i z, \quad \dot{z} = r - C \delta x.$$

No extra prefilter is needed: for step references $r = \text{const}$, the integral channel forces $y \rightarrow r$ (zero steady-state error).

Finally, send the *absolute inner-loop command*

$$\theta_{\text{ref}} = \theta_0 + u.$$

4.3.4 How to pick the LQI weights (a practical recipe)

Use *Bryson's rule* to set diagonal weights so that each state/input is penalized relative to an acceptable maximum magnitude:

Pick admissible scales $(x_{\max}, v_{\max}, z_{\max}, u_{\max})$ for δx_1 (position), δx_2 (velocity), z (integral state), and $u = \delta \theta$ (pitch deviation).

Set

$$Q_a = \text{diag}\left(\frac{1}{x_{\max}^2}, \frac{1}{v_{\max}^2}, \frac{1}{z_{\max}^2}\right), \quad R = \frac{1}{u_{\max}^2}.$$

Intuition:

Larger Q_a entries $\rightarrow$ push states smaller (tighter tracking, potentially larger control).

Larger $R \rightarrow$ be conservative with u (smoother θ , less saturation risk).

A good starting point for outer-loop position control is to emphasize position over velocity

$$q_x \gg q_v, \quad q_z \in [0.2, 1] q_x, \quad R \text{ small but positive.}$$

Then solve the CARE and obtain $[K_x \quad K_i]$. Iterate (Q_a, R) once or twice to meet rise-time and smoothness targets.

4.3.5 Implementation details

Actuator limits & anti-windup for the integral state $\theta_{\max}$ bounds the command θ_{ref} . When saturation occurs, protect the integral channel to avoid windup, e.g. *back-calculation* or *conditional integration* on z :

$$\theta_{\text{ref}} = \text{clip}(\theta_0 + u, -\theta_{\max}, \theta_{\max}), \quad \dot{z} = \begin{cases} r - C\delta x, \\ r - C\delta x + k_{\text{aw}}(\theta_{\text{ref}} - \theta_0 - u), \end{cases}$$

not saturated or helping to desaturate, back-calc.

State availability / observer We measure $y = \delta x_1$. If $\delta x_2 = \dot{x}$ is noisy or not directly measured, use a *state observer* (Luenberger/Kalman):

$$\dot{\hat{x}} = A\hat{x} + Bu + L(y - C\hat{x}), \quad \hat{\xi} = \begin{bmatrix} \hat{x} \\ z \end{bmatrix}, \quad u = -K_x\hat{x} - K_i z.$$

Place observer poles $3 \sim 5 \times$ faster than the outer-loop bandwidth.

Loop separation Keep the inner *attitude/vertical loops* at least $3 \sim 5 \times$ faster than this outer x -loop so that θ behaves as a directly commanded i .

Discretization (if implemented digitally) Discretize (A, B) (e.g., zero-order hold) at sampling time T_s . Implement the integral update for z as

$$z_{k+1} = z_k + T_s(r_k - y_k) \quad (\text{with anti-windup logic when saturated}).$$